

**INSTITUTO POLITÉCNICO NACIONAL**

**ESCUELA SUPERIOR DE CÓMPUTO**




# Programas Clase 28

**Unidad de Aprendizaje:** *Sistemas Distribuidos*

**Profesor:** *Coronilla Contreras Ukranio*

**Alumnos:**

*De la Rosa Gutiérrez Kaleb Yael*

**Grupo***: 7CM2*

```java
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.Signature;
import java.security.spec.PKCS8EncodedKeySpec;
import java.time.Instant;
import java.util.Base64;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

public class GoogleOAuthJWT {

    // Valores por defecto (pueden sobreescribirse pasando
argumentos: <bucket> <objeto> [credentials.json])
    private static final String DEFAULT_BUCKET = "escomsd";
    private static final String DEFAULT_OBJECT = "ergo proxy.jpg";
    private static final String DEFAULT_CREDENTIALS =
"credentials.json";

    public static void main(String[] args) throws Exception {

        String bucketName = (args.length >= 1) ? args[0] :
DEFAULT_BUCKET;
        String objectName = (args.length >= 2) ? args[1] :
DEFAULT_OBJECT;
        String credentialsPath = (args.length >= 3) ? args[2] :
DEFAULT_CREDENTIALS;

        // 1. Leer JSON credentials
        String json = new
String(Files.readAllBytes(Paths.get(credentialsPath)),
StandardCharsets.UTF_8);
```

```
        JsonObject creds =
JsonParser.parseString(json).getAsJsonObject();

        String clientEmail =
creds.get("client_email").getAsString();
        String privateKeyPem =
creds.get("private_key").getAsString();
        String tokenUri = creds.get("token_uri").getAsString();

        // 2. Limpiar clave privada PEM
        privateKeyPem = privateKeyPem
                .replace("-----BEGIN PRIVATE KEY-----", "")
                .replace("-----END PRIVATE KEY-----", "")
                .replaceAll("\\s", "");

        byte[] keyBytes =
Base64.getDecoder().decode(privateKeyPem);

        PKCS8EncodedKeySpec spec = new
PKCS8EncodedKeySpec(keyBytes);
        KeyFactory kf = KeyFactory.getInstance("RSA");
        PrivateKey privateKey = kf.generatePrivate(spec);

        // 3. Crear Header JWT
        String headerJson = "{\"alg\":\"RS256\",\"typ\":\"JWT\"}";
        String header =
base64UrlEncode(headerJson.getBytes(StandardCharsets.UTF_8));

        // 4. Crear Payload JWT
        long now = Instant.now().getEpochSecond();

        String payloadJson = "{"
                + "\"iss\":\"" + clientEmail + "\","
                +
"\"scope\":\"https://www.googleapis.com/auth/devstorage.read_only\
","
                + "\"aud\":\"" + tokenUri + "\","
                + "\"exp\":" + (now + 3600) + ","
                + "\"iat\":" + now
                + "}";

        String payload =
```

```
base64UrlEncode(payloadJson.getBytes(StandardCharsets.UTF_8));

        // 5. Firmar JWT
        String unsignedToken = header + "." + payload;

        Signature signature =
Signature.getInstance("SHA256withRSA");
        signature.initSign(privateKey);

signature.update(unsignedToken.getBytes(StandardCharsets.UTF_8));
        byte[] signed = signature.sign();

        String jwt = unsignedToken + "." +
base64UrlEncode(signed);

        // 6. Enviar a OAuth token endpoint
        String requestBody =
"grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer"
                + "&assertion=" + jwt;

        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(tokenUri))
                .header("Content-Type",
"application/x-www-form-urlencoded")

.POST(HttpRequest.BodyPublishers.ofString(requestBody))
                .build();

        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

        // 7. Mostrar respuesta
        System.out.println("Respuesta:");
        System.out.println(response.body());

        if (response.statusCode() != 200) {
            System.err.println("Error al obtener el token OAuth
2.0 (HTTP " + response.statusCode() + ")");
            return;
        }
```

```
        // 8. Extraer el access_token de la respuesta JSON
        JsonObject tokenResponse =
JsonParser.parseString(response.body()).getAsJsonObject();
        String accessToken =
tokenResponse.get("access_token").getAsString();

        // 9. Construir la peticion GET a Google Cloud Storage
JSON API para descargar el archivo
        String encodedObject = URLEncoder.encode(objectName,
StandardCharsets.UTF_8).replace("+", "%20");
        String storageUrl =
"https://storage.googleapis.com/storage/v1/b/"
                + bucketName + "/o/" + encodedObject +
"?alt=media";

        System.out.println("\nDescargando objeto '" + objectName +
"' del bucket '" + bucketName + "'...");
        System.out.println("URL: " + storageUrl);

        HttpRequest downloadRequest = HttpRequest.newBuilder()
                .uri(URI.create(storageUrl))
                .header("Authorization", "Bearer " + accessToken)
                .GET()
                .build();

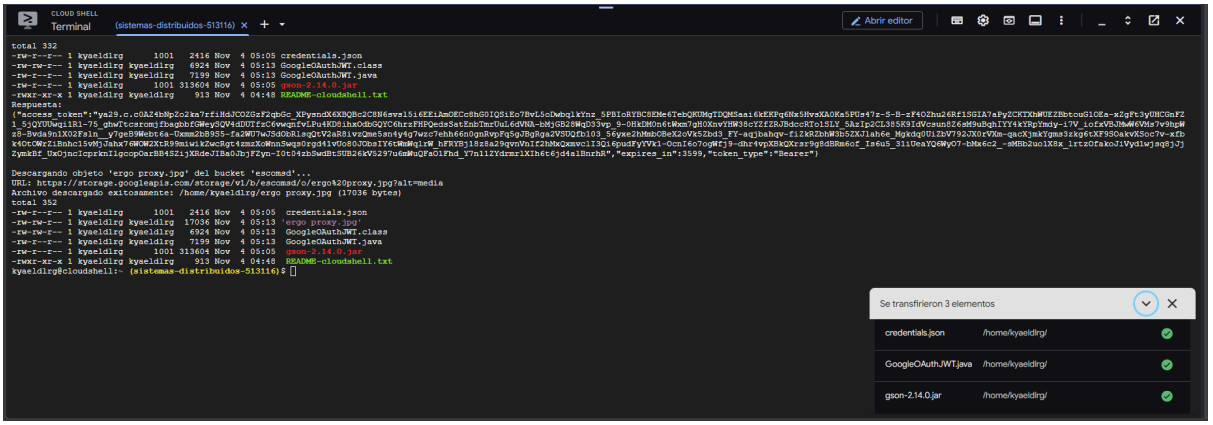
        // 10. Recibir los bytes del objeto y guardarlos en disco
        HttpResponse<byte[]> downloadResponse = client.send(
                downloadRequest,
                HttpResponse.BodyHandlers.ofByteArray());

        if (downloadResponse.statusCode() == 200) {
            String localFileName =
Paths.get(objectName).getFileName().toString();
            Path outputPath = Paths.get(localFileName);
            Files.write(outputPath, downloadResponse.body());
            System.out.println("Archivo descargado exitosamente: "
+ outputPath.toAbsolutePath()
                    + " (" + downloadResponse.body().length + "
bytes)");
        } else {
            System.err.println("Error al descargar el archivo
```

```
(HTTP " + downloadResponse.statusCode() + "):");
            System.err.println(new String(downloadResponse.body(),
StandardCharsets.UTF_8));
        }
    }

    private static String base64UrlEncode(byte[] input) {
        return
Base64.getUrlEncoder().withoutPadding().encodeToString(input);
    }
}
```

# Evidencia


CLOUD SHELL
Terminal
(sistemas-distribuidos-513116)
Abrir editor
total 332
-rw-r--r-- 1 kyaeldlrg 1001 2416 Nov 4 05:05 credentials.json
-rw-rw-r-- 1 kyaeldlrg kyaeldlrg 6924 Nov 4 05:13 GoogleOAuthJWT.class
-rw-r--r-- 1 kyaeldlrg kyaeldlrg 7199 Nov 4 05:13 GoogleOAuthJWT.java
-rw-r--r-- 1 kyaeldlrg 1001 313604 Nov 4 05:05 gson-2.14.0.jar
-rwxr-xr-x 1 kyaeldlrg kyaeldlrg 913 Nov 4 04:48 README-cloudshell.txt
Respuesta:
Descargando objeto 'ergo proxy.jpg' del bucket 'escomsd'...
URL: https://storage.googleapis.com/storage/v1/b/escomsd/o/ergo%20proxy.jpg?alt=media
Archivo descargado exitosamente: /home/kyaeldlrg/ergo proxy.jpg (17036 bytes)
total 352
-rw-r--r-- 1 kyaeldlrg 1001 2416 Nov 4 05:05 credentials.json
-rw-rw-r-- 1 kyaeldlrg kyaeldlrg 17036 Nov 4 05:13 'ergo proxy.jpg'
-rw-rw-r-- 1 kyaeldlrg kyaeldlrg 6924 Nov 4 05:13 GoogleOAuthJWT.class
-rw-r--r-- 1 kyaeldlrg kyaeldlrg 7199 Nov 4 05:13 GoogleOAuthJWT.java
-rw-r--r-- 1 kyaeldlrg 1001 313604 Nov 4 05:05 gson-2.14.0.jar
-rwxr-xr-x 1 kyaeldlrg kyaeldlrg 913 Nov 4 04:48 README-cloudshell.txt
kyaeldlrg@cloudshell:~ (sistemas-distribuidos-513116)$
Se transfirieron 3 elementos
credentials.json /home/kyaeldlrg/
GoogleOAuthJWT.java /home/kyaeldlrg/
gson-2.14.0.jar /home/kyaeldlrg/